

TransformED AI — Sprint 4 Report (Dev 4)

For: Engineering Manager

Sprint: Sprint 4 (Weeks 8–9) — "Stability, tuning, and load testing — no new features"

Report date: 2026-09-07

Prepared by: Dev 4 (WebSocket handlers, login security, the tutor's decision-making logic, check-in guardrails)

Scope: This report covers Dev 4's own Sprint 4 tasks only — not a cross-team report.

1. The one-paragraph version

Sprint 4 was supposed to be a tuning sprint: take everything built in Sprints 1–3 and dial it in using real student sessions. Of my 9 planned tasks, 3 are fully done and shipped, and 6 are sitting in the same spot for the same reason: **there are no real students using the app yet, so there's no real data to tune against.** I wrote down the exact method for each of those 6 pieces of analysis in advance — what to measure, how, and the decision rule to apply — so the moment real sessions exist, the work is a data pull and a formula, not a design exercise. In the meantime I didn't sit idle: I picked up other work that doesn't depend on real students (a production incident fix, a real backend for the tutor Q&A feature, and this sprint's Bug Resolution items). **Below, I've flagged which of the 6 pending items are genuinely stuck waiting for real students, versus which ones could actually move forward right now if we spend a little effort on infrastructure instead of waiting** — see Section 4.

2. Where things stand

#	Task (plain language)	Status	Why
1	Fix the bug where a check-in overlay could get permanently stuck on screen	■ Done	Found, fixed, and safety-netted so it can never happen again even if a future bug reintroduces it
2	Build the "what state is this student's lesson in" admin endpoints	■ Done	27 tests passing, used for demos and support
3	Lock down the exact definition of the 3 camera-based engagement signals with the frontend team	■ Done	All 4 developers signed off; prevents a whole class of "we each assumed something different" bugs
4	Is the "student seems distracted" cutoff score set correctly?	■ Waiting on real students	Method is written; needs ~20 real lesson sessions to compute an answer
5	Are students actually responding to check-in messages, or ignoring them?	■ Waiting on real students + a small piece of logging that isn't built yet	See Section 4 — part of this can move now
6	Is the 2-minute quiet period after a check-in too long or too short?	■ Waiting on real students + the same logging gap as #5	See Section 4 — part of this can move now

#	Task (plain language)	Status	Why
7	Can the system handle 50 students online at once without breaking?	■ Waiting on a deployed test server, not students	Already proven locally — see Section 4
8	If a student's internet drops mid-lesson, do they land back where they left off?	■ Waiting on a deployed test server, not students	Already proven locally — see Section 4
9	Do the check-in messages sound warm and human, not robotic?	■ Waiting on sample lesson content, not students	See Section 4 — no students needed at all

3 of 9 done. 6 of 9 blocked — but only 1 of those 6 is blocked purely on real students with no other path forward.

3. What's actually done, in plain terms

Check-in overlay could get stuck on screen forever. If a student's engagement dropped and a check-in message appeared, there was a path where dismissing it didn't reliably return them to the lesson — they'd be stuck looking at the overlay. This is now fixed two ways: the normal "student dismisses it" path works correctly, *and* a 45-second safety timer automatically clears the overlay even if the dismiss action never arrives (e.g. a flaky connection). Found during this fix and closed in the same pass: a second, sneakier version of almost the same bug that a first-round review caught before it shipped.

Admin visibility into a live session. Support/admin can now look up "what state is this student's tutor in right now" and can manually trigger a check-in for demo or troubleshooting purposes — without needing to watch a live camera feed. 27 automated tests confirm it behaves correctly, including when Redis (our fast in-memory store) is unavailable.

Engagement-signal contract frozen. The three numbers the browser sends every 5 seconds (gaze, expression, interaction) are now precisely defined and agreed by all four developers, including what a `null` value means (camera failed to initialize) — this was previously a fuzzy, undocumented assumption on the frontend side that got corrected during this work.

4. ■■ Heads-up: not everything pending is actually blocked on real students

You asked me to flag this specifically, so here it is up front rather than buried in the tables above.

Genuinely stuck on real students — no other path (1 item):

- **Is the distraction cutoff score (50) set correctly?** This can only be answered by watching real sessions. There's no synthetic substitute — the whole point is calibrating against actual human attention patterns.

Partially unblocked — a piece of this can start today (2 items):

- **Are students responding to check-ins? / Is the 2-minute cooldown right?** Both of these are missing a piece of plumbing that has nothing to do with having real students: right now, when a check-in fires, we don't log *when* it fired or whether the student dismissed it. That logging (a new event type + a small write to our analytics

table) is pure engineering work I can build now. Once it exists, the *analysis* still needs real sessions — but we won't lose another sprint discovering the logging gap after students finally arrive.

Not blocked on students at all — blocked on infrastructure or sample content (3 items):

- **Can we handle 50 concurrent students?** I already built the load-testing tool and proved it works: 50 simulated connections, zero dropped, all check-ins acknowledged, in well under our speed budget. What's actually missing is a real deployed test server to point it at — this is the same India-region hosting migration that's already on the roadmap as a prerequisite for launch, not something that needs real students first. Once that server exists, this can close in an afternoon.
- **Does reconnecting after a dropped connection work?** Same situation — proven for all 7 possible lesson states against a simulated server. What's left is testing against a real, live, deployed server with an actual network interruption, not real users.
- **Do the check-in messages read as warm, not robotic?** This needs 5 real *generated lesson packages* to review actual message text — but "generated" just means running our own content pipeline on a sample textbook chapter. It does not require a single real student. This is waiting on Dev 1 producing sample output, which could happen this week if prioritized.

My recommendation: if we want visible Sprint 4 progress before real students exist, the two highest-leverage moves are (a) spin up the India-region staging server so items 7 and 8 above can close, and (b) ask Dev 1 for 5 sample-generated lesson packages so item 9 can close. Neither needs a single paying or test student.

5. What I worked on instead of waiting

Sprint 4 tuning work being blocked didn't mean idle time. In parallel:

- **Fixed a real production incident** (a 3-hour crash-loop caused by a missing deployment secret) and added an automated check that now blocks any future deploy missing a required secret — closes a gap left over from an earlier fix.
- **Built the real backend for the tutor's Q&A feature** (previously a stub) — a student can now ask the tutor a question mid-lesson and get a real, grounded answer, with rate-limiting and cost controls in place. 26 new automated tests.
- **Started Bug Resolution sprint work** that specifically doesn't depend on real students — adapting caption timing to work with real human-recorded narration instead of fixed-length text-to-speech, and closing a login security gap (bot-verification not yet checked on the backend).

6. Bottom line

Sprint 4's tuning goals are exactly where they should be for a product with zero real students so far: **not "behind"** — **legitimately blocked**, with every method pre-written so the actual analysis is fast once data exists. One item (the distraction-score calibration) has no way around waiting for real students. The other five have a real, nameable unblock that isn't "wait for users": two need a small logging addition, two need a staging server that's already on our roadmap for other reasons, and one needs sample content Dev 1 can generate without a single student. If it's useful, I can start on the logging piece and the staging-server-dependent tests as soon as that server exists — just say the word.

Source: [docs/dev4-tracker.md](#) — *Sprint 4 section*